

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the output?

```
try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

- A)B C
- B)A
- C)A C
- D)B

Q2. What will happen?

```
def fun(n):
    if n == 0:
        return 1
    return n * fun(n)
```

```
print(fun(3))
```

- A)6
- B)3
- C)TypeError
- D)RecursionError

Q3. What happens when this code runs?

```
def first():
    second()
```

```
def second():
    third()
```

```
def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

- A)Handled
- B)RuntimeError cannot propagate through functions
- C)NameError
- D)TypeError

Q4. What will be printed?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        print(x)
```

```
    x = 30
    inner()
```

```
outer()
```

- A)20
- B)NameError
- C)30
- D)10

Q5. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

- A)Finally only
- B)10 only
- C)Finally followed by 10
- D)10 followed by Finally

Q6. What will be printed?

```
try:
    x = int("10")
except ValueError:
    print("A")
```



else:

print("B")

finally:

print("C")

A)A

B)B

C)B C

D)A C

Q7. Suppose sample.py contains:

x = 10

def display():

print("Hello")

What will this print?

import sample

print(sample.x)

sample.display()

A)Error

B)Only Hello

C)10

Hello

D)x

display

Q8. What is the output?

try:

raise ValueError

except ValueError:

print("A")

raise

A)A, followed by propagation of the same exception

B)B

C)No Output

D)A, and execution ends normally

Q9. What will be printed?

try:

print(10 / 0)

except ZeroDivisionError:

print("A")

except Exception:

print("B")

A)A B

B)Nothing

C)B

D)A

Q10. What will be the output?

x = 10

def fun():

print(x)

x = 20

fun()

A)UnboundLocalError

B)10

C)20

D)NameError

Q11. Suppose module_a.py contains:

print("Module loaded")

value = 10

And another file contains:

import module_a

import module_a

print(module_a.value)

Assuming both imports occur in the same Python process, what is normally printed?

A)ImportError

B)10

10

C)Module loaded

10

D)Module loaded

Module loaded

10

Q12. What is the output?

f = lambda x: x + 5

print(f(10))

A)5

B>Error

C)10



D)15

Q13. What happens here?

```
def fun(a):  
    return a
```

```
print(fun())
```

A)None

B)NameError

C)0

D)TypeError

Q14. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

```
print(fun())
```

A)A

C

D

20

B)A

B

10

D

C)A

B

D

10

D)A

B

C

D

10

Q15. What will happen?

```
def outer():
```

x = 10

```
def inner():  
    nonlocal y  
    y = 20
```

```
inner()
```

```
outer()
```

A)SyntaxError

B)10

C)20

D)NameError

Q16. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)30

B)TypeError

C)SyntaxError

D)20

Q17. What is the output?

```
def fun(x):  
    return x * 2
```

```
a = fun  
print(a(5))
```

A>Error

B)func(5)

C)10

D)5

Q18. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:
```



```

    print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)

```

- A)Exception
- Age is too low
- B)InvalidAgeError
- 15
- C)InvalidAgeError
- Age is too low
- D)Nothing, because custom exceptions cannot be caught

Q19. What will be printed?

```

from functools import reduce

```

```

numbers = [1, 2, 3, 4]

```

```

result = reduce(
    lambda a, b: a * b,
    numbers
)

```

```

print(result)

```

- A)10
- B)0
- C)24
- D)9

Q20. What is the output?

```

def fun(x):
    x += 10
    return x

```

```

x = 5
y = fun(x)

```

```

print(x, y)

```

- A)15 5
- B)5 5
- C)5 15
- D)15 15

Q21. What is the purpose of this statement?

```

if __name__ == "__main__":

```

A)It prevents syntax errors

B)It identifies code intended to run when the file is executed directly

C)It creates the main function

D)It prevents all imports

Q22. What is the output?

```

def outer():

```

```

    x = 10

```

```

    def inner():
        return x + 5

```

```

    return inner

```

```

f = outer()

```

```

print(f())

```

- A)15
- B)NameError
- C)10
- D)None

Q23. What will normally happen?

```

import math

```

```

import math

```

- A)The module is normally loaded and executed twice
- B)The module is normally loaded once and reused
- C)SyntaxError
- D)ImportError

Q24. What happens here?

```

from math import sqrt

```

```

print(sqrt(36))

```

- A)6
- B)NameError
- C)36
- D)6.0

Q25. What is the output?

```

def fun(x):

```

```

    if x == 0:

```

```

        return

```

```

    return x + fun(x - 1)

```

```

print(fun(3))

```

- A)5



B)3

C)None

D)6

Q26. What will be the output?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        global x
```

```
        x = 30
```

```
    inner()
```

```
    print(x)
```

```
outer()
```

```
print(x)
```

A)20

10

B)20

30

C)30

30

D)30

20

Q27. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)NameError

B)10

C)20

D)None

Q28. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

A)A

B)A B

C)SyntaxError

D)B

Q29. What happens when this code executes?

```
def fun():
```

```
    print(variable)
```

```
    variable = 50
```

```
variable = 10
```

```
fun()
```

A)10

B)NameError

C)UnboundLocalError

D)50

Q30. What will be printed?

```
def fun(a, b):
```

```
    return a + b
```

```
print(fun(10, b=20))
```

A)TypeError

B)SyntaxError

C)30

D)20

Q31. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
```

```
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)B

B)A

C)Nothing

D)A and B

Q32. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
```



```

map(
    lambda x: x * 2,
    filter(lambda x: x % 2 == 0,
numbers)
)
)

```

print(result)

A)[2, 4, 6, 8, 10]

B)[2, 4]

C)[4, 8]

D)[1, 3, 5]

Q33. What happens here?

try:

```

    raise ValueError("Wrong value")

```

except ValueError as e:

```

    print(e)

```

A)Wrong Value

B)ValueError

C)Nothing

D)e

Q34. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

A)random cannot be imported

B)Python automatically renames the local file

C)The local file can shadow the intended standard-library module

D)Python will always reject it

Q35. What happens here?

```

class MyError(Exception):
    pass

```

def test():

```

    raise MyError("Something went wrong")

```

try:

```

    test()

```

except MyError as e:

```

    print(type(e).__name__)

```

A)TypeError

B)Something went wrong

C)MyError

D)Exception

Q36. Why is

try:

```

    print(10 / 0)

```

except Exception:

```

    print("A")

```

except ZeroDivisionError:

```

    print("B")

```

codes's second handler never reached?

A)ZeroDivisionError occurs before Exception

B)ZeroDivisionError is not an exception

C)Exception is a superclass of ZeroDivisionError

D)except blocks execute randomly

Q37. What happens when this function is called?

```

def fun(a=10, b):

```

```

    return a + b

```

A)It executes correctly

B)TypeError

C)SyntaxError

D)NameError

Q38. What is the output?

```

def fun(a=10, b=20):

```

```

    return a - b

```

```

print(fun(30))

```

A)30

B)-10

C)20

D)10

Q39. Which object keeps track of modules that have already been imported during the current Python process?

A)__module__

B)sys.imports

C)sys.path

D)sys.modules

Q40. What is the output?

```

def outer():

```



x = 10

```
def inner():  
    nonlocal x  
    x = x + 5  
    return x
```

```
inner()  
return x
```

print(outer())

- A)5
- B)10
- C)15
- D)UnboundLocalError

Q41. What will be the output?

import math

print(math.sqrt(25))

- A)5
- B)sqrt(25)
- C)5.0
- D)25

Q42. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

- A)20
- B)10
- C)None
- D)RuntimeError

**Q43. if test.py has
print("A")**

```
if __name__ == "__main__":  
    print("B")
```

**If test.py is executed directly, what
will be printed?**

- A)Nothing
- B)A followed by B
- C)B
- D)A

Q44. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

- A)cleanup
100
- B)100
cleanup
- C)cleanup
-5
- D)negative
cleanup

Q45. What will be printed?

```
def make_counter():  
    count = 0  
  
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

return counter

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```

- A)1
2
3
4
- B)1
1
1



C)0

1

0

2

D)1

2

1

3

Q46. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

A)B

B)A B

C)A

D)Nothing

Q47. What is the major difference between the following?

import math

and

from math import sqrt

A)First imports only sqrt

B)There is no difference

C)First provides the module namespace; second imports sqrt directly

D)Both import only sqrt

Q48. What does the following function return?

```
def fun():  
    print("Hello")
```

x = fun()

print(x)

A>Error

B>Hello

C>Hello None

D)None

Q49. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

a = fun()

b = fun()

print(a)

print(b)

A>Error

B)[10]

[10, 10]

C)[10, 10]

[10]

D)[10]

[10]

Q50. What will happen?

```
def fun(a, b=10):  
    return a + b
```

print(fun(b=5, a=20))

A)SyntaxError

B)TypeError

C)15

D)25

